

$$- \ell_x(\mu, \Sigma) = - \log \prod_{i=1}^{N_K} p(x_i | \mu, \Sigma)$$

$$= - \sum_{i=1}^{N_K} \log p(x_i | \mu, \Sigma)$$

$$= - \sum_{i=1}^{N_K} \log \left(\frac{1}{\sqrt{2\pi} |\Sigma|} e^{-\frac{1}{2} (x_i - \mu)^T \Sigma^{-1} (x_i - \mu)} \right)$$

$$= - \sum_{i=1}^{N_K} \left[\log \left(\frac{1}{\sqrt{2\pi} |\Sigma|} \right) - \frac{1}{2} (x_i - \mu)^T \Sigma^{-1} (x_i - \mu) \right]$$

$$= \sum_{i=1}^{N_K} \left[\log(\sqrt{2\pi} |\Sigma|) + \frac{1}{2} (x_i - \mu)^T \Sigma^{-1} (x_i - \mu) \right]$$

• Matrix differentiation rules:

$$1) \frac{\partial}{\partial \mu} (x - \mu)^T \Sigma (x - \mu) = -2 \Sigma (x - \mu)$$

$$2) \frac{\partial}{\partial A} \log |A| = (A^T)^{-1}$$

• Derivative w.r.t. μ (for each class separately)

$$\frac{\partial}{\partial \mu} \log(\pi |\Sigma| \mu, \Sigma) = \frac{\partial}{\partial \mu} \left(\sum_{i=1}^{N_K} \frac{1}{2} (x_i - \mu)^T \Sigma^{-1} (x_i - \mu) \right)$$

$$= - \sum_{i=1}^{N_K} \Sigma^{-1} (x_i - \mu) \stackrel{!}{=} 0 \quad (\text{mult. by } \Sigma)$$

$$\Rightarrow - \sum_{i=1}^{N_K} (x_i - \mu) \stackrel{!}{=} 0$$

$$\Rightarrow -\sum_{i=1}^{N_K} x_i + \sum_{i=1}^{N_K} \mu = 0$$

$$\Rightarrow N_K \mu_K = \sum_{i=1}^{N_K} x_i$$

$$\Rightarrow \boxed{\hat{\mu}_K = \frac{1}{N_K} \sum_{i=1}^{N_K} x_i}$$



• Derivative w.r.t. Σ_K (for each class)

$K: \Sigma^{-1}$ (precision matrix)

$$z_i := x_i - \mu$$

$$S: \sum_{i=1}^N z_i z_i^T = \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T$$

scatter matrix $\in \mathbb{R}^{D \times D}$

(dropped all K 's :))

$$\frac{\partial}{\partial K_K} \log(\mathcal{L} | \mu_K, \Sigma_K) = \frac{\partial}{\partial K} \left(\sum_{i=1}^N \log \left(\underbrace{\sqrt{\det(2\pi K)^{-1}}}_{\det(2\pi K)} \right) + \frac{1}{2} (x_i - \mu)^T K (x_i - \mu) \right)$$

inserted
for
clarity!
remainder
remains
the same

$$= \frac{\partial}{\partial K} \left(\sum_{i=1}^N -\frac{1}{2} \log \det(2\pi K) + \frac{1}{2} z_i^T K z_i \right)$$

$$= \frac{\partial}{\partial K} \left(\sum_{i=1}^N -\frac{1}{2} \log(2\pi)^D - \frac{1}{2} \log \det(K) + \frac{1}{2} z_i^T K z_i \right)$$

$$\stackrel{(2)}{=} \sum_{i=1}^N -\frac{1}{2} (K^T)^T + \frac{1}{2} z_i z_i^T \stackrel{!}{=} 0$$

$$\Rightarrow \sum_{i=1}^N -\frac{1}{2} K^{-1} + \underbrace{\frac{1}{2} \sum_{i=1}^N z_i z_i^T}_{=S} \stackrel{!}{=} 0$$

$$= - (K^{-1}) \sum_{i=1}^N 1 + S \stackrel{!}{=} 0$$

$$\Rightarrow S = K^{-1} \cdot N = \Sigma \cdot N$$

$$\Rightarrow \hat{\Sigma} = \frac{1}{N} S = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T$$

2.8.3 Training and Prediction with QDA:

1) Training: For each class k , compute:

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{i: y_i = k} x_i \quad \hat{\Sigma}_k = \frac{1}{N_k} \sum_{i: y_i = k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$$

2) Prediction:

For a new instance x , compute the posterior:

$$\hat{y} = \underset{k}{\operatorname{argmax}} p(y=k|x)$$

$$= \underset{k}{\operatorname{argmax}} \frac{p(x|y=k) p(y=k)}{p(x)}$$

$$= \underset{k}{\operatorname{argmin}} \left[-\log p(x|y=k) - \log p(y=k) \right]$$

prior class prob = $\frac{N_k}{N}$

N_k = instances in class in TS ; N = total number of instances

$$= \underset{k}{\operatorname{argmin}} \left[\frac{1}{2} \log \det (2\pi \hat{\Sigma}_k) + \frac{1}{2} (x - \hat{\mu}_k)^T \hat{\Sigma}_k^{-1} (x - \hat{\mu}_k) - \log p(y=k) \right]$$

sample estimates

independent of $x \rightarrow$ put into b_k

$$b_k := \frac{1}{2} \log \det (2\pi \hat{\Sigma}_k) - \log p(y=k)$$

QDA decision rule:

$$\hat{y} = \underset{k}{\operatorname{argmin}} \left[\frac{1}{2} (x - \hat{\mu}_k)^T \hat{\Sigma}_k^{-1} (x - \hat{\mu}_k) + b_k \right]$$

squared Mahalanobis distance

Eucld. dist. $d = \sqrt{(x - \mu)^T (x - \mu)}$

Recap GDA

$$p(y=k|x) = \frac{\overbrace{p(x|y=k)}^{N(\mu_k, \Sigma_k)} \overbrace{p(y=k)}^{\approx \frac{N_k}{N}}}{p(x)}$$

$$p(y=k|x) \propto N(\mu_k, \Sigma_k) \frac{N_k}{N}$$

LINEAR DISCRIMINANT ANALYSIS (LDA)

- Assumption: All classes share the same covariance $\Sigma_k = \Sigma_w$

- Training: $\hat{\Sigma}_w = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_{y_i})(x_i - \mu_{y_i})^T$

mean of class assigned to y_i

- Rewrite decision rule using $\hat{K}_w = \hat{\Sigma}_w^{-1}$:

$$\begin{aligned} \hat{y} &= \underset{k}{\operatorname{argmin}} \left[\frac{1}{2} (x - \hat{\mu}_k)^T \hat{K}_w (x - \hat{\mu}_k) + b_k \right] \\ &= \underset{k}{\operatorname{argmin}} \left[\frac{1}{2} \underbrace{(x^T \hat{K}_w x - \mu_k^T \hat{K}_w x - x^T \hat{K}_w \mu_k + \mu_k^T \hat{K}_w \mu_k)}_{\text{indep. of } k} + b_k \right] \\ &= \underset{k}{\operatorname{argmin}} \left(\underbrace{-\mu_k^T \hat{K}_w x}_{:= w_k^T} + \underbrace{b_k}_{:= \tilde{b}_k} \right) \end{aligned}$$

$$= \underset{k}{\operatorname{argmax}} (w_k^T x - \tilde{b}_k)$$

- Example of $C = 2$, the the decision rule will reduce to the following:

- 2 classes: "1", "0"

$$w_1, \tilde{b}_1 \text{ and } w_0, \tilde{b}_0 :$$

$$\hat{y} = \underset{k=0,1}{\operatorname{argmax}} (w_k^T x - \tilde{b}_k)$$

$$w_1^T x - \tilde{b}_1 \geq w_0^T x - \tilde{b}_0 \Rightarrow \hat{y} = 1$$

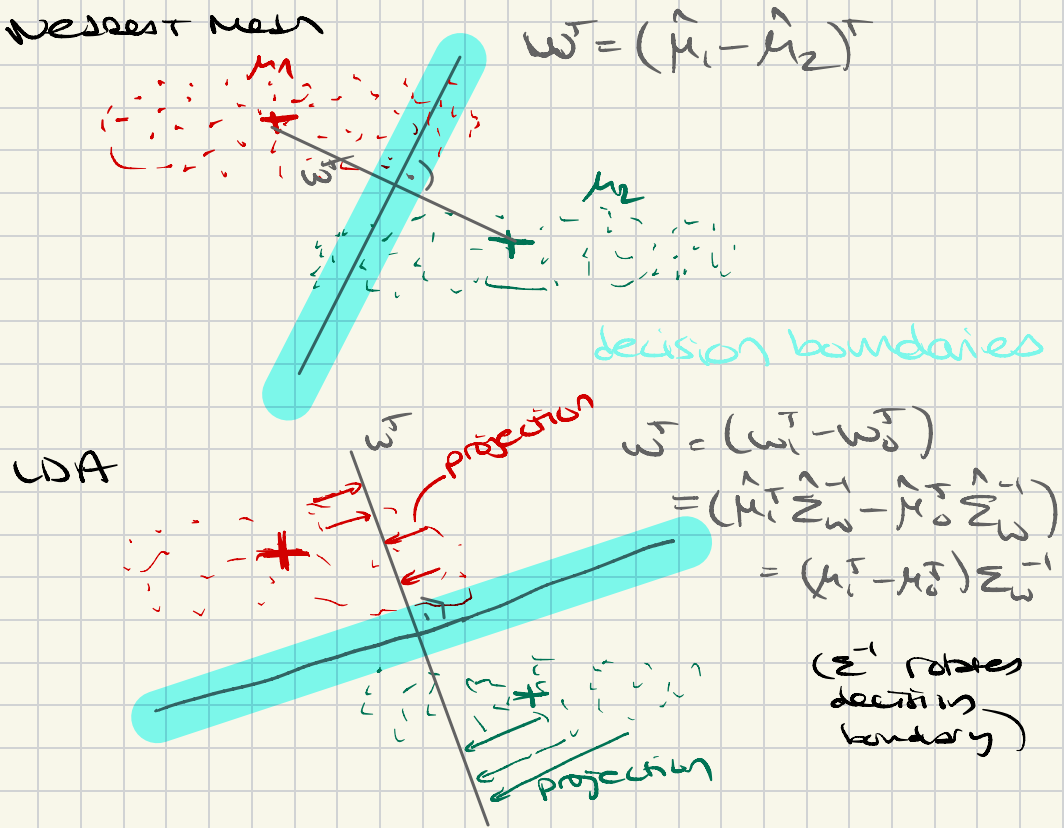
$$w_1^T x - \tilde{b}_1 - (w_0^T x - \tilde{b}_0) \geq 0 \Rightarrow \hat{y} = 1$$

$$\Rightarrow \underbrace{(w_1^T - w_0^T)}_{:= \tilde{w}^T} x - \underbrace{(\tilde{b}_1 - \tilde{b}_0)}_{:= \tilde{b}} \geq 0 \left. \begin{array}{l} \Rightarrow \hat{y} = 1 \\ < 0 \Rightarrow \hat{y} = 0 \end{array} \right\}$$

- obtain linear classifier:

$$\hat{y} = \operatorname{sign}(w^T x + \tilde{b})$$

- COMPARISON OF NEAREST MEAN CLASS. TO LDA



2.9 LOGISTIC REGRESSION

- linear classifier
- outputs posterior probs. $p(y=k|x)$
- core idea: Transform the generative model into an analytic expression for the posterior (assuming LHS of Bayes rule)

$$\hat{y} = \underset{k}{\operatorname{argmax}} p(y=k|x)$$

- assume centered data and balanced classes:

$$N_{-1} = N_{+1} = \frac{N}{2}$$

under these assumptions:

$$\mu_{-1} = -\mu_{+1}$$

Proof:

$$\begin{aligned} \mu &= \frac{1}{N} \sum_i x_i \\ &= \frac{1}{2N_{-1}} \sum_{i: y_i = -1} x_i + \frac{1}{2N_{+1}} \sum_{i: y_i = +1} x_i \\ &= \frac{1}{2} \mu_{-1} + \frac{1}{2} \mu_{+1} \stackrel{\text{! by mean centering}}{=} 0 \\ \mu_{-1} &= -\mu_{+1} \end{aligned}$$

2.3.1

Derivation of Analytic Formula for posterior

$$p(y=1) = p(y=-1) \quad \text{by balanced classes}$$

- start w/ Bayes rule

$$p(y=k|x) = \frac{p(x|y=k)p(y=k)}{p(x)}$$

- for 2 classes:

$$p(y=1|x) = \frac{p(x|y=1)p(y=1)}{\sum_k p(x|y=k)p(k)}$$

$$= \frac{p(x|y=1) \cancel{p(y=1)}}{\cancel{p(x|y=1) \cancel{p(y=1)}} + \cancel{p(x|y=-1) \cancel{p(y=-1)}}}$$

$$= \frac{p(x|y=1)}{p(x|y=1) + p(x|y=-1)} \quad \left(\because p(x|y=1) \right)$$

$$= \frac{1}{1 + \frac{p(x|y=-1)}{p(x|y=1)}}$$

ass $p(x|y=k) = N(\mu_k, \Sigma_w)$

$$p(y=1|x) = \frac{1}{1 + \exp(-x^T w)} \quad \left. \begin{array}{l} \text{sigmoid} \\ \text{function} \end{array} \right\}$$

$$\frac{p(x|y=-1)}{p(x|y=1)} = \frac{\frac{1}{\sqrt{\det(2\pi\Sigma_w)}} e^{-\frac{1}{2}(x-\mu_{-1})^T \Sigma_w^{-1} (x-\mu_{-1})}}{\frac{1}{\sqrt{\det(2\pi\Sigma_w)}} e^{-\frac{1}{2}(x-\mu_{+1})^T \Sigma_w^{-1} (x-\mu_{+1})}}$$

$$= \frac{\exp\left(-\frac{1}{2}(x-\mu_{-1})^T \Sigma_w^{-1} (x-\mu_{-1})\right)}{\exp\left(-\frac{1}{2}(x-\mu_{+1})^T \Sigma_w^{-1} (x-\mu_{+1})\right)}$$

$$= \frac{\exp\left(-\frac{1}{2}(x^T \Sigma_w^{-1} x - 2x^T \Sigma_w^{-1} \mu_{-1} + \mu_{-1}^T \Sigma_w^{-1} \mu_{-1})\right)}{\exp\left(-\frac{1}{2}(x^T \Sigma_w^{-1} x - 2x^T \Sigma_w^{-1} \mu_{+1} + \mu_{+1}^T \Sigma_w^{-1} \mu_{+1})\right)}$$

$$\frac{\exp(x)}{\exp(y)} = \exp(x-y)$$

$$= \exp(-\cancel{\frac{1}{2} x^T \Sigma_w^{-1} x} + x^T \Sigma_w^{-1} \mu_{-1} - \cancel{\frac{1}{2} \mu_{-1}^T \Sigma_w^{-1} \mu_{-1}} - (-\cancel{\frac{1}{2} x^T \Sigma_w^{-1} x} + x^T \Sigma_w^{-1} \mu_{+1} - \cancel{\frac{1}{2} \mu_{+1}^T \Sigma_w^{-1} \mu_{+1}}))$$

$$= \exp(x^T \Sigma_w^{-1} \mu_{-1} - \cancel{\frac{1}{2} \mu_{-1}^T \Sigma_w^{-1} \mu_{-1}} - x^T \Sigma_w^{-1} \mu_{+1} + \cancel{\frac{1}{2} \mu_{+1}^T \Sigma_w^{-1} \mu_{+1}})$$

$$= \exp(x^T \Sigma_w^{-1} (\mu_{-1} - \mu_{+1}))$$

$$= \exp(-x^T \underbrace{\Sigma_w^{-1}}_{= w} (\mu_{+1} - \mu_{-1}))$$

NOTE:

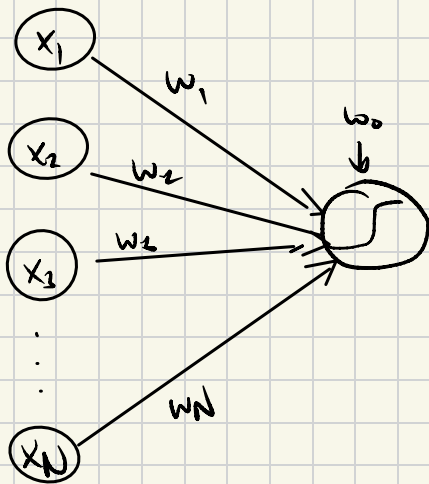
$$\begin{aligned} \mu_{-1}^T \Sigma_w^{-1} \mu_{-1} &= (-\mu_{+1})^T \Sigma_w^{-1} (-\mu_{+1}) \\ &= \mu_{+1}^T \Sigma_w^{-1} \mu_{+1} \end{aligned}$$

$\Rightarrow$ logistic regression assumes a logistic function for the class label posterior; parametrized by 2 weight vector w

$\Rightarrow$ we learn w directly from our data!



"logistic sigmoid function"



$$z = x^T w \begin{cases} > 0 & \text{vote class 1} \\ < 0 & \text{vote class -1} \end{cases}$$

Properties of the sigmoid

1) Property:

$$\text{if } p(y=+1|x) = \sigma(w^T x)$$

due to normalization of probabilities, then we also have

$$\begin{aligned} p(y=-1|x) &= 1 - p(y=+1|x) \\ &= 1 - \sigma(w^T x) \\ &= \sigma(-w^T x) \end{aligned}$$

2) Property: $\frac{\partial \sigma(z)}{\partial z} = \sigma'(z)$

$$= \sigma(z)(1 - \sigma(z))$$

$$b'(z) = \frac{\partial}{\partial z} \frac{1}{1 + e^{-z}}$$

$$= -1 (1 + e^{-z})^{-2} (-1) e^{-z}$$

$$= \frac{1}{1 + e^{-z}} \frac{1 \cdot e^{-z}}{1 + e^{-z}}$$

$$= \frac{1}{1 + e^{-z}} \cdot \frac{1}{1 + e^z}$$

$$= b(z)b(-z)$$

2.2 Derivation of Objective function of Logistic Regression

Maximum likelihood estimation

- we want to estimate the parameters:

$$\hat{w} = \underset{w}{\operatorname{argmax}} p(y_1, \dots, y_N | x_1, x_2, \dots, x_N; w)$$

- we assume our data to be i.i.d:

$$\begin{aligned}
 p_{\theta}(y_1, \dots, y_N | x_1, \dots, x_N) &= \prod_{i=1}^N \underbrace{p(y_i | x_i)}_{\text{ber}(\pi(x_i))} \\
 &= \prod_{i: y_i=1}^N p(y_i=1 | x_i) \cdot \prod_{i: y_i=0}^N p(y_i=0 | x_i) \\
 &= \prod_{i=1}^N p(y_i=1 | x_i)^{y_i} p(y_i=0 | x_i)^{(1-y_i)}
 \end{aligned}$$

Binary classif.
 $y_i \in \{0, 1\}$

• Take the negative logarithm:

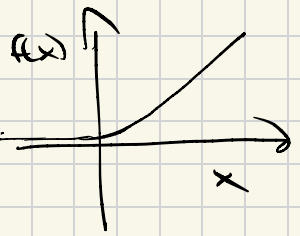
$$\begin{aligned}
 -\log p(y_1, \dots, y_N | x_1, \dots, x_N) &= \left[\sum_{i=1}^N y_i \log \overbrace{p(y_i=1 | x_i)}^{\mathcal{B}(x_i^T w)} \right. \\
 &\quad \left. + (1-y_i) \log \underbrace{p(y_i=0 | x_i)}_{\mathcal{B}(-x_i^T w)} \right]
 \end{aligned}$$

Optimization objective of Logistic Regression:

$$\hat{w} = \underset{w}{\operatorname{argmin}} \left[\sum_{i=1}^N (y_i \log \mathcal{B}(x_i^T w) + (1-y_i) \log \mathcal{B}(-x_i^T w)) \right]$$

Remarks

(1) $\text{softplus}(x) = \log(1 + e^x)$



For $y_i \in \{-1, 1\}$, and $z = x^T w$,

then we can rewrite our loss function as

$$l(\hat{y}_i, y_i^*) = \text{softplus}(-y_i z)$$

(2) Derivation of loss for logistic regression corresponds to MLE for iid Bernoulli random variables $y_i \sim \text{Ber}(\pi(x_i))$.

2.13 Derivative of the objective:

$$\begin{aligned} & \frac{\partial}{\partial w} (1 - y_i) \log \sigma(-x_i^T w) \\ &= (1 - y_i) \frac{1}{\cancel{\sigma(-x_i^T w)}} \underbrace{b'(-x_i^T w)}_{\cancel{\sigma(-x_i^T w)} \sigma(x_i^T w)} (-x_i) \\ &= (1 - y_i) \sigma(x_i^T w) (-x_i) \end{aligned}$$

$$\frac{\partial}{\partial \omega} y_i \log b(x_i^T \omega)$$

$$= y_i \frac{1}{b(x_i^T \omega)} b(x_i^T \omega) b(-x_i^T \omega) x_i$$

$$= y_i b(-x_i^T \omega) x_i$$

$$\begin{aligned} \frac{\partial}{\partial \omega} \ell(\omega) &= -\sum_i \left((1-y_i) b(x_i^T \omega) (-x_i) + \right. \\ &\quad \left. y_i \underbrace{b(-x_i^T \omega) x_i}_{= 1 - b(x_i^T \omega)} \right) \end{aligned}$$

$$= -\sum_i \left(b(x_i^T \omega) (-x_i) - y_i b(x_i^T \omega) (-x_i) + y_i x_i - y_i b(x_i^T \omega) x_i \right)$$

$$= -\sum_i \left(b(x_i^T \omega) (-x_i) + y_i x_i \right)$$

$$= \sum_i \left((b(x_i^T \omega) - y_i) x_i \right) \stackrel{!}{=} 0$$

Optimization by Gradient Descent

- initial guess $w^{(0)} = 0$

- For $t = 1 \dots T$ ($T = \text{number of iterations}$)

$$\begin{aligned} w^{(t+1)} &= w^{(t)} - \tau \frac{\partial}{\partial w} L(w^{(t)}) \\ &= w^{(t)} - \tau \left(\sum_{i=1}^N \underbrace{((b(x_i^T w) - y_i^*) x_i)}_{\text{error}} \right) \end{aligned}$$

How does the derivative behave?

- Case 1: $y_i^* = 1, x_i^T w \rightarrow \infty, b(x_i^T w) \approx 1$
 $\Rightarrow \text{error} \approx 0$; no adjustments req.

- Case 2: $y_i^* = 0, x_i^T w \rightarrow -\infty, b(x_i^T w) \approx 0$
 $\Rightarrow \text{error} \approx 0$; no adjust.

- Case 3: $y_i^* = 1, b(x_i^T w) \approx 0$
error: $b(x_i^T w) - 1 = 0 - 1 = -1$
 $\Rightarrow w \leftarrow w + x_i \tau$

- Case 4: $y_i^* = 0, b(x_i^T w) \approx 1$
 $\Rightarrow \text{error} : 1 - 0 = 1$
 $\Rightarrow w \leftarrow w - \tau \cdot x_i$

Variations of Gradient Descent:

1) Mini-batch descent:

- include only random subset of training instances (in the sum term)
- Choose B (set size) uniformly at random $\hat{=}$ mini-batch

2) Stochastic Gradient Descent

- $B = 1$ (increase T)

2.10 SUMMARY OF LINEAR CLASSIFICATION

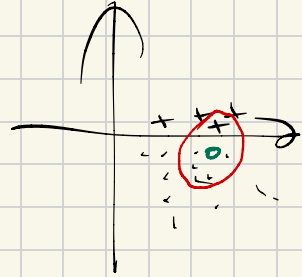
° Distinguish between parametric and non-parametric models

• Parametric models

- have fixed, finite set of parameters regardless of data
- make some functional assumptions
- typically faster, less prone to overfitting

• non-parametric models

- make fewer assumptions
- become arbitrarily complex w/ data
- ^{typically} more flexible at the cost of overfitting



linear and non-linear

Generative

Discriminative

Parametric

LDA/QDA

linear
logistic reg., SVM,
Perceptron

Non-
Parametric

Density
Trees

k-NN, Kernel SVMs

Decision Trees

LINEAR CLASSIFIERS AND COMMON OBJECTIVE

$$\hat{w}, \hat{b} = \underset{w, b}{\operatorname{argmin}} \quad \frac{1}{N} \sum_{i=1}^N \operatorname{loss}(y_i^*, x_i^T w + b) + \underbrace{\frac{\lambda}{2} w^T w}_{\text{regularization}}$$

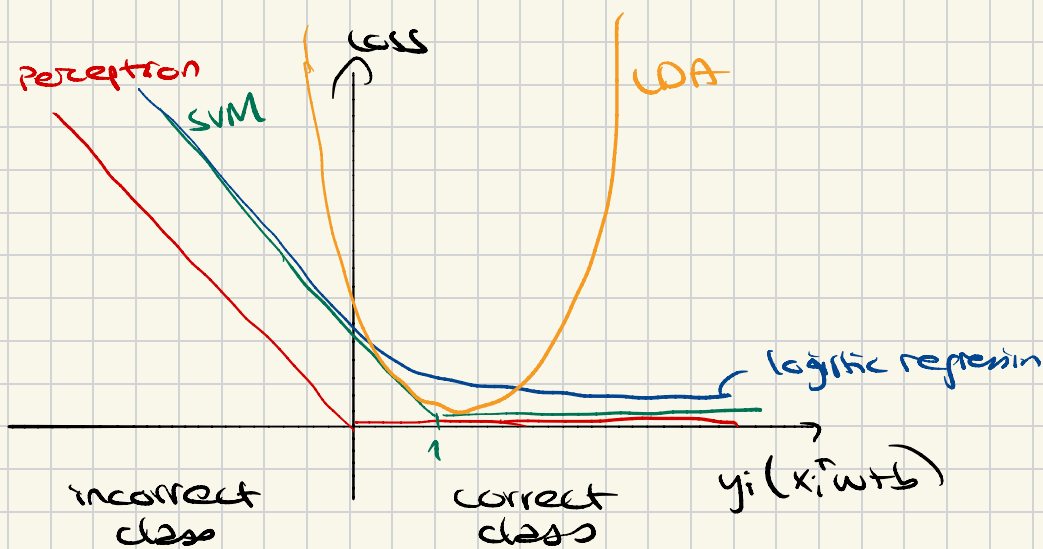
LINEAR Methods	loss	regularization
Perceptron	$\operatorname{ReLU}(-y_i^*(x_i^T w + b))$	$\lambda = 0$
SVM	$\operatorname{ReLU}(1 - y_i^*(x_i^T w + b))$	$\lambda > 0$
LOGISTIC REGRESSION	$\operatorname{softplus}(-y_i^*(x_i^T w + b))$	$\lambda = 0, \lambda > 0$
LDA*	$(y_i - (x_i^T w + b))^2$	$\lambda = 0$

* Remark: $(x_i, y_i), i=1-N, y \in \{0, 1\}$,
assume $x_i | y_i \sim N(\mu_k, \Sigma)$ and class

priors are estimated, then the decision boundary of log. reg. coincides with

that of solving a least squares problem

$$\min_{w, b} \sum_i (w^T x + b - y_i)^2.$$



2.11 MULTI-CLASS CLASSIFICATION

- Problem: $C > 2$
- Approaches:
 - 1) Reduce the problem to a series of 2 class problem
 - 2) Use an algo. that can naturally deal w/ multiple classes
- Three methods:
 - 1) one-against-rest classification
 - 2) All pairs classification
 - 3) Joint classific. via softmax

1) One - against - Rest

- For each class $k = 1 \dots C$, train a binary classifier
- Training: For each k , treat instances with $y_i^* = k$ as the positive class (+1), and all others ($y_i \neq k$) as negative (-1)
- We compute scores

$$s_k = x_i^T w_k + b_k$$

To make one scores comparable across classifiers, we normalize:

$$\tilde{w}_k = \frac{w_k}{\|w_k\|} \quad \tilde{b}_k = \frac{b_k}{\|w_k\|}$$

$$\Rightarrow s_k(x_i) = \tilde{w}_k^T x_i + \tilde{b}_k$$

- Decision Rule:

$$\hat{y} = \begin{cases} \text{"unknown"} & s_k < \epsilon \forall k \\ \underset{k}{\operatorname{argmax}} s_k & \text{otherwise} \end{cases}$$

2) All-Pairs Classification

- For every pair of classes k and k' ($k \neq k'$), train a binary classifier
- in total $\frac{C(C-1)}{2}$ classifiers
- For each pair (k, k') , compute score

$$S_{kk'} = x_i^T W_{kk'} + b_{kk'}$$

- Voting scheme:

- If $S_{kk'} > 0$, assign one vote for ^{class} k
- If $S_{kk'} < 0$, assign k'

- Final decision:

$$\hat{y}_i = \begin{cases} \text{label with the most votes} \\ \text{"uncertain" if votes are} \\ \text{roughly equal} \end{cases}$$

3) Softmax function for joint multi-class classification

- For each class k , we compute a score

$$s_k = \mathbf{w}_k^T \mathbf{x}_k + b_k$$

- Define the posterior prob. for class k using the softmax function:

$$p(\hat{y}_i = k | \mathbf{x}_i) = \frac{\exp(s_k)}{\sum_{k'=1}^C \exp(s_{k'})}$$

$$\vec{s} = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_C \end{bmatrix} = \mathbf{W} \mathbf{x}_i + \vec{b}$$

- In the two class case, softmax reduces to sigmoid fun'n
- Train model using the cross-entropy loss:

$$\text{loss}(y_i^*, \hat{y}_i) = \sum_{k=1}^C \mathbb{1}[k = y_i^*] (-\log p(\hat{y}_i = k | \mathbf{x}_i))$$

CONTINUE: NONLINEAR CLASSIFICATION

Approaches to solve / deal w/ Nonlin. Classific.

1) Use linear model, but increase the feature space

- "Cover's theorem": in higher-dim spaces, data tend to be more easily separable
- Careful: with very high D relative to N overfitting is a risk (when $D+1 = N$, separation is always ~~do~~ possible)
- In these cases, we can use sparse learning methods (LASSO $\rightarrow$ covered later)

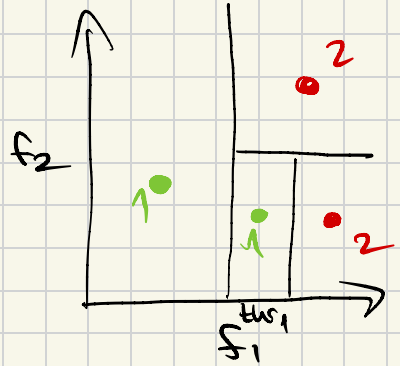
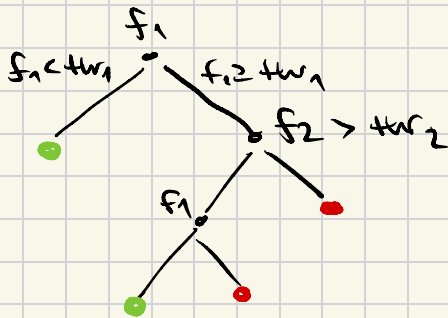
2) Use a nonlinear classifier:

a) QDA

b) Decision trees and random forests

- partition recursively the feature space and can model complex nonlin. decision boundaries

tree 1 ... T trees



$$x_i = \begin{pmatrix} f_1 \\ f_2 \end{pmatrix}$$

3) Nonlinear Transformation of features

- transform original features x_i using a non-linear mapping

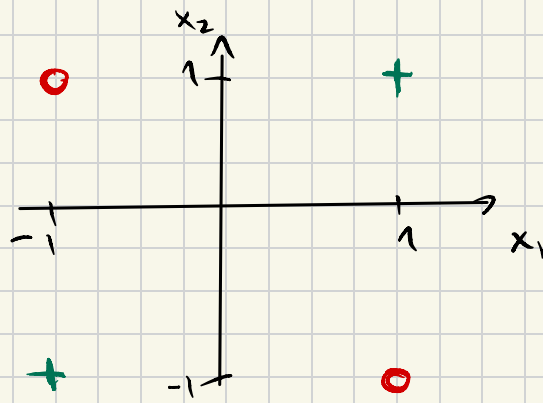
$$\tilde{x}_i = \phi(x_i) \quad \rightarrow \text{nonlin. func}$$

Example 1

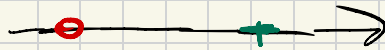
- original features $x_1, x_2 \in \{-1, 1\}$

- True response:

$$y^* = \begin{cases} +1 & \text{if } x_1 \neq x_2 \\ -1 & \text{if } x_1 = x_2 \end{cases}$$



- transformed features $\tilde{x} = x_1 \cdot x_2$



- Example 2: BMI

$$BMI = \frac{\text{weight}}{(\text{height})^2}$$

Challenge: manually crafting the transformation $\phi(x)$ is time-consuming and requires expert knowledge

Solution: learn $\phi(x)$ automatically

- Kernel-SVM (uses a linear classifier on nonlin. transformed features)
- using multi-layer neural networks